C# 14 und Ausblick auf C# 15

From productivity gains to type-safe paradigm shifts

Christian Nagel

https://www.cninnovation.com

Christian Nagel

- 20+ years of experience with C# and .NET
- Author of 20+ books on C# and .NET (latest: "Pragmatic Microservices with C# and Azure")
- Microsoft MVP for C# and .NET
- Speaker at global conferences (including BASTA! for 25+ years)
- Passionate about language design and developer productivity

Agenda

1. **C# 13 Recap** — Key features connecting to 14 & 15
2. **C# 14 Deep Dive** — Released with .NET 10
3. **C# 15 Preview** — What's coming with .NET 11
4. **Evolution Patterns** — How the language grows
5. **Adoption Strategy** — What to use today

Building the bridge to C# 14 and C# 15
C# 13
C# 14
C# 15
Part 1: C# 13 in Review

C# 13 — params Collections

Extended beyond arrays — now accepts any collection type

```
// C# 12 and earlier – only arrays
void Log(params string[] messages) { }

// C# 13 – any collection type
void Log(params IEnumerable<string> messages) { }
void Log(params ReadOnlySpan<string> messages) { }  // zero-alloc!
void Log(params List<string> messages) { }

// Call site looks identical
Log("Starting", "Processing", "Done");
```

Why it matters for C# 14+: Paves the way for **implicit Span conversions** and zero-allocation APIs.

C# 13 — Partial Properties

partial keyword extended to properties

```
// Declaration (e.g., in designer file)
public partial class UserViewModel
{
  public partial string Name { get; set; }
  public partial int Age { get; set; }
}
```

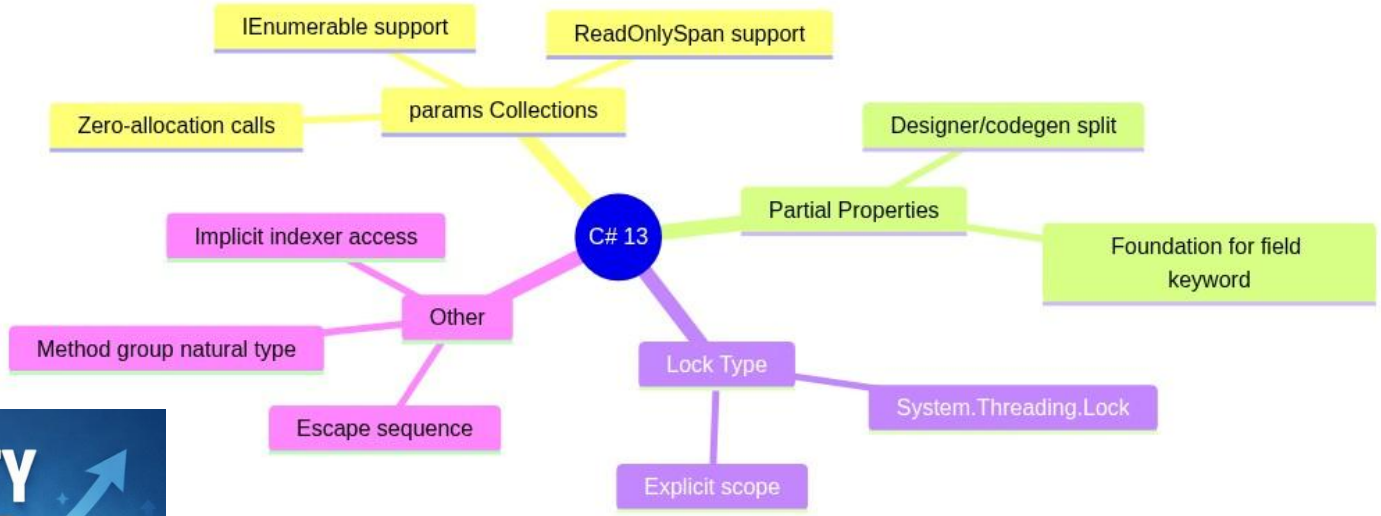
Foundation for: field keyword in C# 14 and source generator patterns.

C# 13 — Lock Type

New System.Threading.Lock — replaces lock(object)

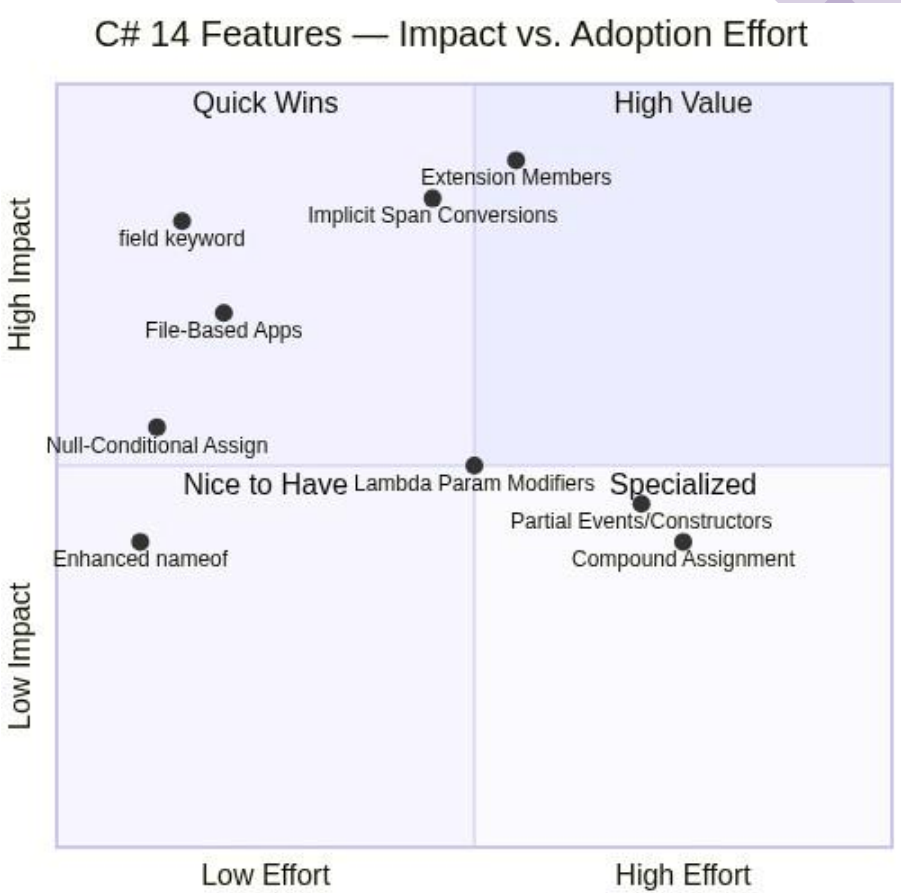
```
// Before C# 13
private readonly object _lock = new();
lock (_lock) { /* critical section */ }

// C# 13 – dedicated Lock type
private readonly Lock _lock = new();
lock (_lock) { /* critical section */ }

// Lock also supports explicit scope
using (_lock.EnterScope())
{
  // critical section
} // automatically released
```

Benefit: Better diagnostics, IDE support, and thread-safety improvements.



C# 13 Feature Summary

C# 14 Feature Overview



field Keyword (1)

Direct access to the compiler-generated backing field

```
// Before C# 14 — manual backing field
public class User
{
  private string _name = string.Empty;
  public string Name
  {
    get => _name;
    set => _name = value?.Trim()
      ?? throw new ArgumentNullException(nameof(value));
  }
}
```

field Keyword (2)

```
// C# 14 — field keyword (zero overhead)
public class User
{
  public string Name
  {
    get;
    set => field = value?.Trim()
      ?? throw new ArgumentNullException(nameof(value));
  }

  // Lazy loading with null-coalescing assignment
  public List<string> Roles
  {
    get => field ??= LoadRolesFromDatabase();
  }

  // Value clamping with default initializer
  public int Volume
  {
    get;
    set => field = Math.Clamp(value, 0, 100);
  } = 50;
}
```

field Keyword — INotifyPropertyChanged

Tip: Source generators (e.g., MVVM Toolkit) use partial + field to eliminate all boilerplate.

Enhanced nameof

Support for unbound (open) generic types

```
// Before C# 14 — had to provide type arguments
string name1 = typeof(List<int>).Name.Split('`')[0]; // "List"

// C# 14 — clean and safe
string name2 = nameof(List<>);          // "List"
string name3 = nameof(Dictionary<,>);   // "Dictionary"

// Real-world: Logging and diagnostics
public void RegisterService<T>()
{
  _logger.LogInformation(
    "Registering service: {ServiceType}",
    nameof(IRepository<>));
}

// Attribute usage
[Display(Name = nameof(IEnumerable<>))]
public class GenericList { }
```

File-Based Apps (dotnet run app.cs)

Run C# without a project file — .NET 10

```
dotnet run hello.cs
```

```
// hello.cs — no .csproj needed!
Console.WriteLine("Hello, .NET 10!");

var now = DateTimeOffset.UtcNow;
Console.WriteLine($"Running at {now:HH:mm:ss}");
```

- ✅ Top-level statements — no Main, no namespace boilerplate
- ✅ Full .NET runtime and NuGet ecosystem available
- ✅ IntelliSense in VS Code with C# Dev Kit
- ✅ Migrate when ready: dotnet project convert hello.cs

File-Based Apps — Directives (1)

Add NuGet packages and alternate SDKs inline:

```
// api.cs
#:sdk Microsoft.NET.Sdk.Web
#:package Microsoft.AspNetCore.OpenApi@10.0.0

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddOpenApi();
var app = builder.Build();

app.MapGet("/", () => "Hello from a single file!");
app.MapGet("/time", () => DateTimeOffset.UtcNow);

app.Run();
```

```
dotnet run api.cs
```

File-Based Apps — Directives (2)

Directive	Purpose
#:package Name@Version	Add a NuGet package reference
#:sdk SdkName	Use alternate SDK (e.g., Web, Worker)
#:property Name=Value	Set an MSBuild property

File-Based Apps - She-Bang with Linux

```
#!/usr/bin/env dotnet
Console.WriteLine("Hello from a she-bang script!");
```

```
chmod +x hello.cs
```

Extension Members

- ✅ Methods
- ✅ Properties
- ✅ Indexers
- ✅ Operators

Extension Members

```
public static class StringExtensions
{
  extension (string input)
  {
    // Extension property — no parentheses needed
    public int WordCount
      => input.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;

    // Extension method (as before)
    public string Truncate(int maxLength)
      => input.Length <= maxLength ? input : input[..maxLength] + "…";

    // Boolean property
    public bool IsValidEmail
      => Regex.IsMatch(input, @"^[^@\s]+@[^@\s]+\.[^@\s]+$",
          RegexOptions.IgnoreCase);
  }
}
```

Extension Members Usage

```
// Usage
string text = "Hello World from C# 14";
Console.WriteLine(text.WordCount);        // 5 (property!)
Console.WriteLine(text.Truncate(10));     // Hello Wor…
Console.WriteLine("user@example.com".IsValidEmail);  // True (property!)
```

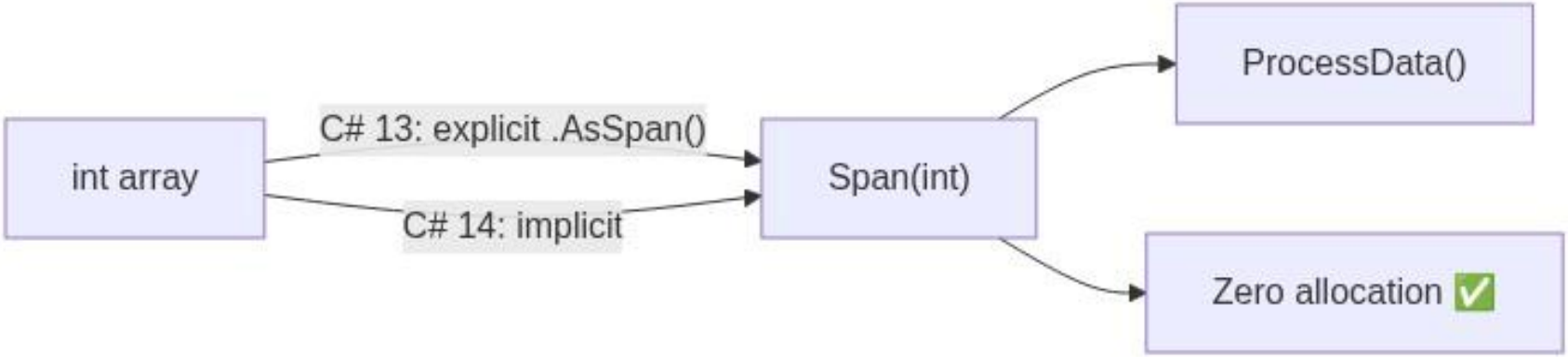
Implicit Span Conversions (1)

Arrays automatically convert to Span<T> / ReadOnlySpan<T>

```
// C# 14 — no explicit conversion needed
void ProcessData(ReadOnlySpan<int> data)
{
  foreach (var item in data)
    Console.Write(item);
}

int[] numbers = [1, 2, 3, 4, 5];
ProcessData(numbers);           // C# 14: implicit conversion ✅

// OverloadResolutionPriority — prefer Span overload
[OverloadResolutionPriority(1)]
void Process(ReadOnlySpan<byte> data) { /* fast path */ }
void Process(byte[] data) => Process((ReadOnlySpan<byte>)data);
```

Implicit Span Conversions (2)

Performance story:



Null-Conditional Assignment

Use ?. on the left side of an assignment

```
// Before C# 14
if (config != null)
  config.Timeout = 30;

// C# 14 — concise and safe
config?.Timeout = 30;

// Works with index access
cache?["key"] = computedValue;

// Chaining
request?.Headers?.["X-Correlation-Id"] = correlationId;

// Real-world example
public void UpdateSettings(AppConfig? config, UserPrefs? prefs)
{
  config?.Theme = prefs?.Theme ?? "Dark";
  config?.Language = prefs?.Language ?? "en";
  config?.Timeout = 30;
}
```

Lambda Parameter Modifiers

Use ref, out, in without explicit type annotations

```
// C# 14 — modifiers without types in lambdas
int[] data = [1, 2, 3, 4, 5];

// ref parameter in lambda — modifies in place
data.ForEach((ref item) => item *= 2);
// data is now [2, 4, 6, 8, 10]

// With scoped modifier for safety (can't escape the lambda)
Span<int> span = [10, 20, 30];
span.Process((scoped ref item) => item += 1);
```

Partial Events & Constructors

```
// Declaration file (e.g., generated)
public partial class OrderProcessor
{
  public partial OrderProcessor(string connectionString);
  public partial event EventHandler<OrderEventArgs>? OrderPlaced;
  public partial event EventHandler<OrderEventArgs>? OrderFailed;
}

// Implementation file (your code)
public partial class OrderProcessor
{
  private readonly string _connectionString;

  public partial OrderProcessor(string connectionString)
  {
    _connectionString = connectionString
      ?? throw new ArgumentNullException(nameof(connectionString));
    Initialize();
  }

  public partial event EventHandler<OrderEventArgs>? OrderPlaced;
  public partial event EventHandler<OrderEventArgs>? OrderFailed;
}
```

Key use case: Source generators can declare constructors/events; you implement the details.

User-Defined Compound Assignment

Custom +=, -=, *=, /= operators

```
public readonly struct Vector2D(double x, double y)
{
  public double X { get; } = x;
  public double Y { get; } = y;

  // Regular operator
  public static Vector2D operator +(Vector2D a, Vector2D b)
    => new(a.X + b.X, a.Y + b.Y);

  // C# 14 — compound assignment (avoids boxing/copying)
  public static Vector2D operator +=(Vector2D current, Vector2D delta)
    => new(current.X + delta.X, current.Y + delta.Y);

  public static Vector2D operator *=(Vector2D v, double scalar)
    => new(v.X * scalar, v.Y * scalar);
}

// Usage
Vector2D position = new(0, 0);
position += new Vector2D(3, 4);   // Uses optimized += operator
position *= 2.0;                  // Uses optimized *= operator
```

C# 14 Summary (1)

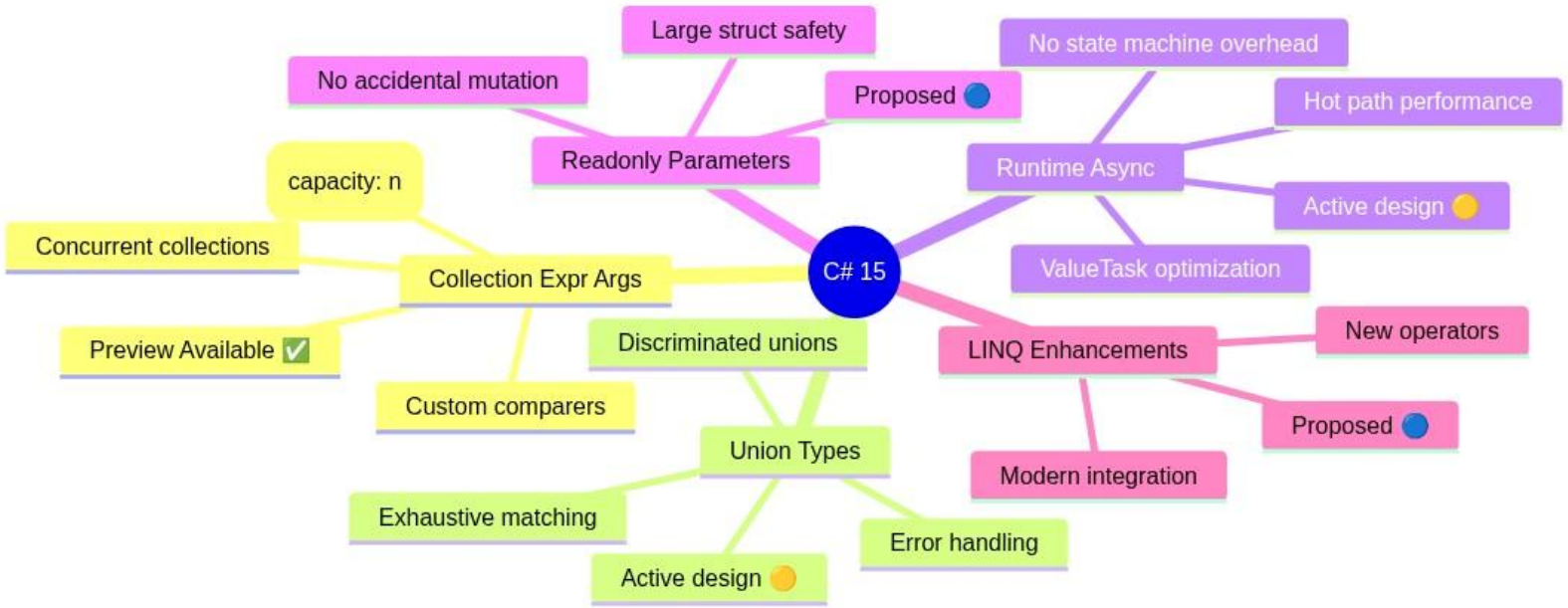
Feature	Impact	Use When
File-Based Apps	🔥 High	Scripts, prototypes, quick experiments
Extension Members	🔥 High	Building fluent APIs, extending types
field Keyword	🔥 High	Adding validation/logic to properties
Implicit Span	🔥 High	Performance-critical data processing

C# 14 Summary (2)

Feature	Impact	Use When
Null-Conditional Assign	Medium	Optional config/conditional updates
Enhanced nameof	Medium	Logging, diagnostics, reflection
Lambda Modifiers	Medium	Large struct performance lambdas
Partial Events/Ctors	Specialized	Source generator scenarios
Compound Assignment	Specialized	Domain-specific value types

Part 3: C# 15 Preview

C# 15 at a Glance



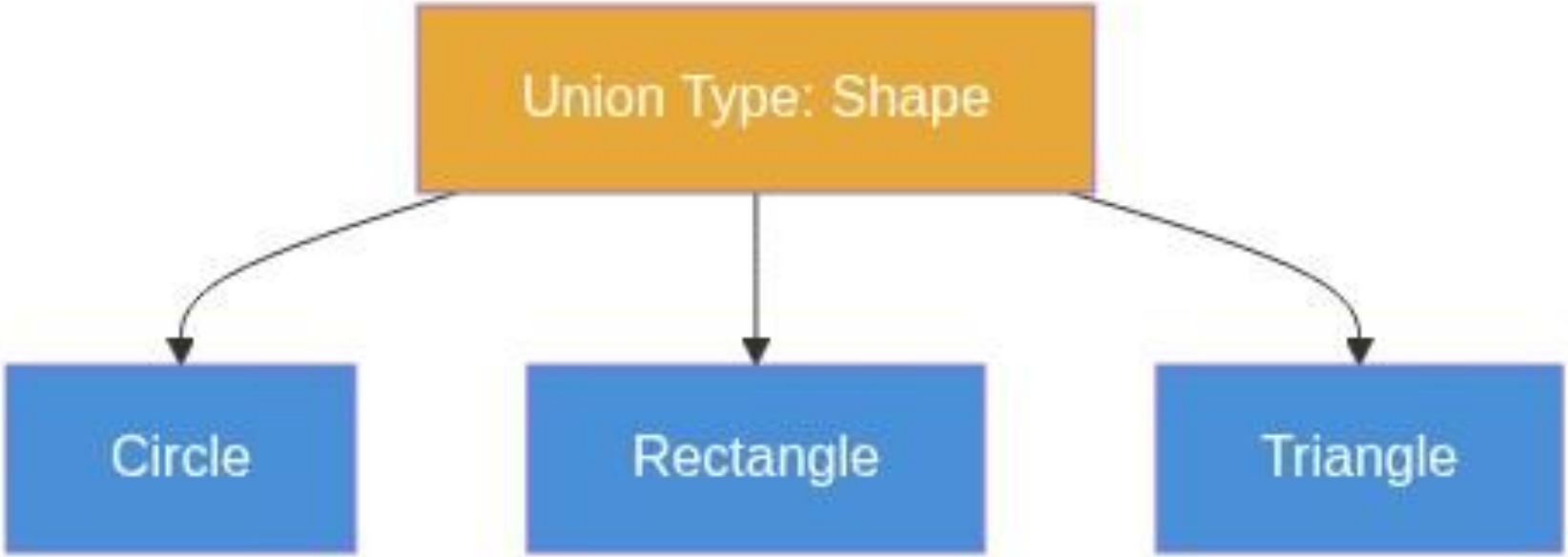
Collection Expression Arguments

Pass constructor arguments inside collection expressions

```
// Before C# 15 — two-step initialization
List<string> names = new(capacity: users.Length * 2);
names.AddRange(users.Select(u => u.Name));

// C# 15 — single expression with capacity hint
List<string> names = [
  with(capacity: users.Length * 2),
  ..users.Select(u => u.Name),
  ..groups.SelectMany(g => g.Members)
];

// Case-insensitive HashSet in one expression
HashSet<string> usernames = [
  with(StringComparer.OrdinalIgnoreCase),
  "Alice", "Bob", "ALICE"   // Only 2 entries — "ALICE" is duplicate
];

// Dictionary with comparer + spreading
Dictionary<string, UserProfile> profiles = [
  with(StringComparer.OrdinalIgnoreCase),
  ..defaultProfiles,
  ["admin"] = adminProfile
];
```

What Are Union Types? (1)

A value that can be one of several distinct types



What Are Union Types? (2)

Traditional C# approach (inheritance):

```
// Base class - any type can inherit
abstract class Shape { }
class Circle : Shape { }
class Rectangle : Shape { }
// ❌ Problem: Can't enforce exhaustiveness - new types can be added
```

C# 15 Union approach:

```
// Closed set - exactly these types, no more
public union Shape(Circle, Rectangle, Triangle);
// ✅ Compiler enforces: handle ALL cases, no surprises
```

Why Unions? The Problem They Solve (1)

Problem: Representing mutually exclusive states

```
// ❌ Before: nullable fields lead to invalid states
public class ApiResponse
{
  public User? Data { get; set; }          // Either data...
  public string? ErrorMessage { get; set; } // ...or error
  public int? ErrorCode { get; set; }       // But which is valid?
}

// Invalid state is possible:
var bad = new ApiResponse
{
  Data = user,            // Has data
  ErrorMessage = "Error" // AND an error? 🤔
};
```

Why Unions? The Problem They Solve (2)

Solution: Unions make invalid states impossible

```
// ✅ C# 15: Can only be ONE of these
public union ApiResponse(Success, Error);
public record Success(User Data);
public record Error(string Message, int Code);

// Compiler prevents invalid states!
```

Union Types: Cross-Language Perspective (1)

Language	Union Syntax	Feature
TypeScript	type Result = Success | Error	Type unions
F#	type Result = Success of User | Error of string	Discriminated unions
Rust	enum Result<T> { Ok(T), Err(String) }	Algebraic data types
Swift	enum Result { case success(User), error(String) }	Enums with associated values
C# 15	union Result(Success, Error)	Discriminated unions ✨

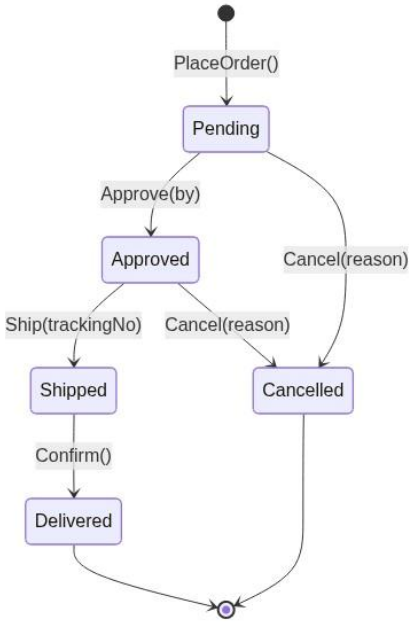
Union Types: Cross-Language Perspective (2)

What C# unions provide:

- ✅ **Type safety** — compiler knows all possible cases
- ✅ **Exhaustiveness checking** — must handle every variant
- ✅ **Pattern matching** — natural integration with switch expressions
- ✅ **Zero invalid states** — can't be in two states at once

Union Types — State Machine

```
public union OrderState(Pending, Approved,
Shipped, Delivered, Cancelled);

public record Pending(DateTime OrderDate);
public record Approved(DateTime Date, string
ApprovedBy);
public record Shipped(string TrackingNumber,
DateTime ShipDate);
public record Delivered(DateTime
DeliveryDate);
public record Cancelled(string Reason);
```



Union Types — Error Handling Without Exceptions

```
// C# 15: union return type — no exceptions needed
public union GetUserResult(Found, NotFound, Unauthorized);
public record Found(User User);
public record NotFound(string UserId);
public record Unauthorized(string Reason);

public GetUserResult GetUser(string id, ClaimsPrincipal caller)
  => (!caller.IsAuthenticated)
    ? new Unauthorized("Login required")
    : _db.Find(id) is User u
      ? new Found(u)
      : new NotFound(id);

// Caller handles all outcomes — compiler enforced
var response = GetUser(id, HttpContext.User) switch
{
  Found(var user)         => Ok(user),
  NotFound(var userId)    => NotFound($"No user {userId}"),
  Unauthorized(var msg)   => Unauthorized(msg)
};
```

Readonly Parameters

Prevent accidental mutation of parameters

```
// Problem: Large struct passed by reference can be accidentally mutated
public void Process(ref LargeStruct data)
{
  data.Value = 42;  // Bug! Caller's value is changed
}

// C# 15 — readonly parameter: read-only, no copy
public void Render(readonly Matrix4x4 transform)
{
  // transform.M11 = 1f;  // Compilation error ✅
  float scale = transform.M11; // Reading is fine
  _renderer.ApplyTransform(transform);
}

// Works with ref too (combines with existing ref readonly)
public double DotProduct(
  readonly ref Vector3 a,
  readonly ref Vector3 b)
  => a.X * b.X + a.Y * b.Y + a.Z * b.Z;
```

Async Methods — The Problem

Optimize async methods by eliminating state machine overhead

```
// Traditional async — always generates
state machine
public async Task<User> GetUserAsync(string
id)
{
  var user = await _cache.GetAsync(id);  //
State machine generated
  return user ?? await _db.GetAsync(id); //
Even if cache hits!
}
```

The problem:

- 🔴 State machine allocation (~200+ bytes)
- 🔴 State tracking overhead
- 🔴 Slower execution even when await completes synchronously

Runtime Async Optimization

C# 15 — Runtime async optimization:

- Async moves from libraries to the runtime
- Compiler automatically optimizes hot paths that complete synchronously!

State machine generated only when actually needed

Runtime Async — Performance Impact

Scenario	.NET 9 (Before)	.NET 10 w/ Runtime Async (After)	Improvement
1M await Task.Yield() calls	~1.25s, 35 MB allocated	~0.95s, 18 MB allocated	**~24% faster, ~49% fewer allocations**
500k await ValueTask completions	~0.82s, 12 MB allocated	~0.63s, 0 MB allocated	**~23% faster, allocations eliminated**
Async file I/O loop (100k ops)	~1.10s, 28 MB allocated	~0.88s, 15 MB allocated	**~20% faster, ~46% fewer allocations**

C# 15 — Feature Status

Feature	Status	.NET 11
Collection Expression Args	Preview	Preview 1+
Union Types	Active design	Likely
Runtime Async	Active design	.NET 10 Preview
Readonly Parameters	In design	Possible
LINQ Enhancements	Proposed	TBD
Expression Tree Extensions	Proposed	TBD

Adoption Strategy

Adoption Strategy: Start Now (C# 14 — .NET 10)

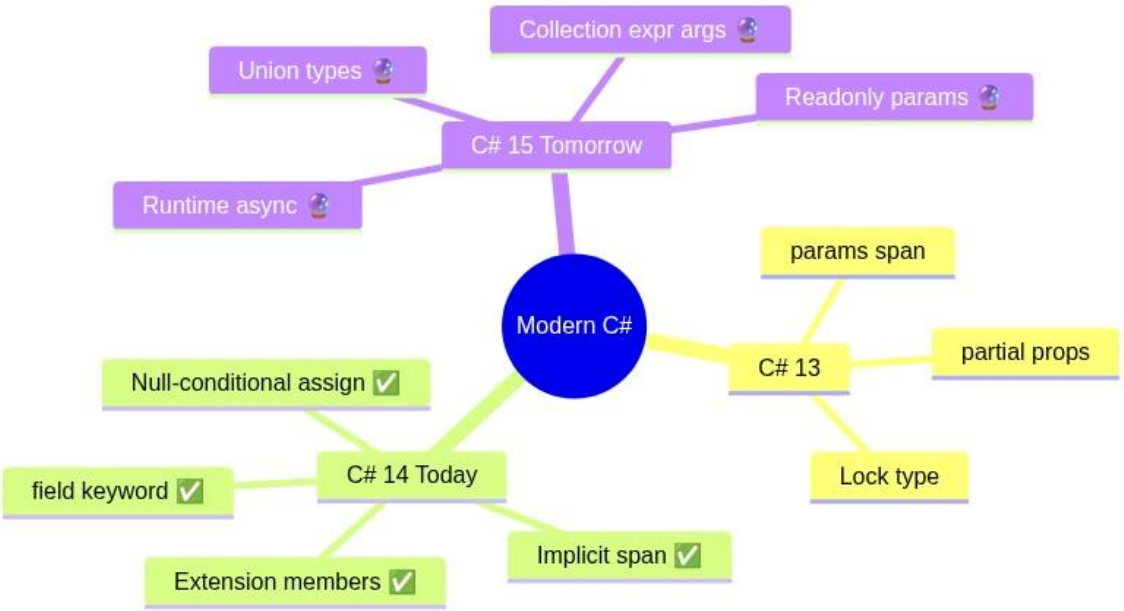
```
// ✅ field keyword — instant win, zero risk
public string Name { get => field; set => field = value.Trim(); }

// ✅ Null-conditional assignment — drop-in replacement
config?.Timeout = 30;

// ✅ Enhanced nameof — wherever you use typeof().Name.Split()
string name = nameof(List<>);
```

Evaluate & Adopt

```
// 🟡 Extension members — refactor helper classes over time
public static extension DateExtensions for DateTime
{
  public bool IsWeekend => DayOfWeek is Saturday or Sunday;
}
// 🟡 Implicit span — in performance hotspots
void Process(ReadOnlySpan<byte> data) { }
```

⚠️ Try using ***Async Runtime*** – don’t use it in production!

Summary (1)


Collection expr args 🔮
Union types 🔮
Readonly params 🔮
C# 15 Tomorrow
Runtime async 🔮
params span
Modern C#
C# 13
partial props
Null-conditional assign ✅
C# 14 Today
Lock type
field keyword ✅
Implicit span ✅
Extension members ✅

Summary (2)

Key takeaways:

- C# grows **incrementally** — each version builds on the last
- C# 14 brings **practical** productivity improvements (adopt today!)
- C# 15 brings **type safety** paradigm shifts (prepare your thinking)

Q & A

Common questions:

- *"Is field just syntax sugar?"* — Yes, zero overhead. Same IL as manual backing fields.
- *"Should we use spans everywhere?"* — No. Only where performance matters.
- *"When will union types ship?"* — Likely C# 15 / .NET 11 (Nov 2026).
- *"Are these breaking changes?"* — No. All features are opt-in.
- *"field is a keyword — what if I have a variable named field?"* — Use @field to escape.

Thank You!

Enjoy BASTA! Spring 2026

Happy coding with C# 14 and beyond!

BASTA! Spring 2026

Slides and samples: **github.com/CNILearn/bastaspring2026**

Wir bitten um
dein Feedback!

BITTE BEWERTEN SIE UNS!

entwickler.de